

A wizard, ray-marched

Rendering with distance functions, in Futhark, on WebGPU

Techniques for programmers who haven’t done graphics

What we built



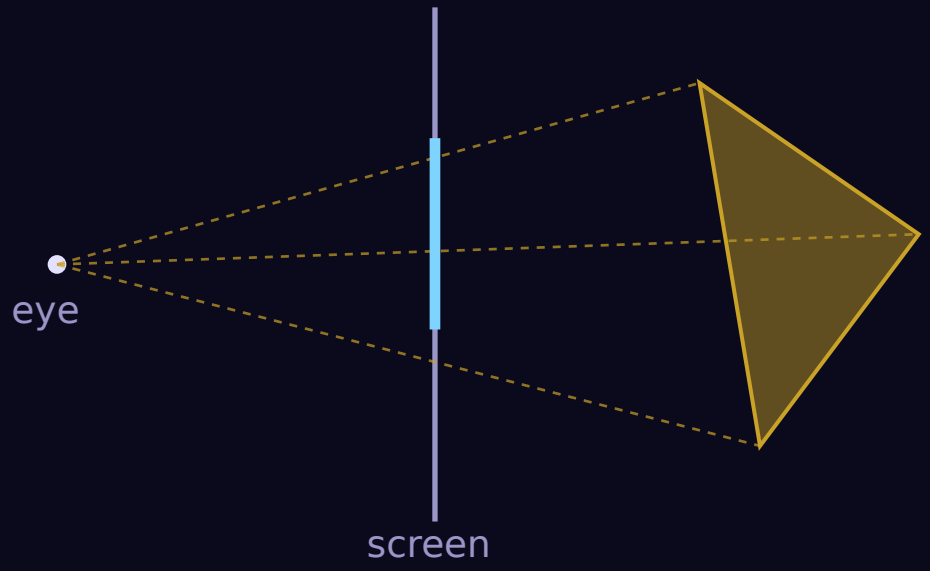
Every pixel is a **pure function** of (time, camera, parameters, game state).

No meshes, no textures, no engine: the wizard, the beard, the terrain, the clouds are all maths.

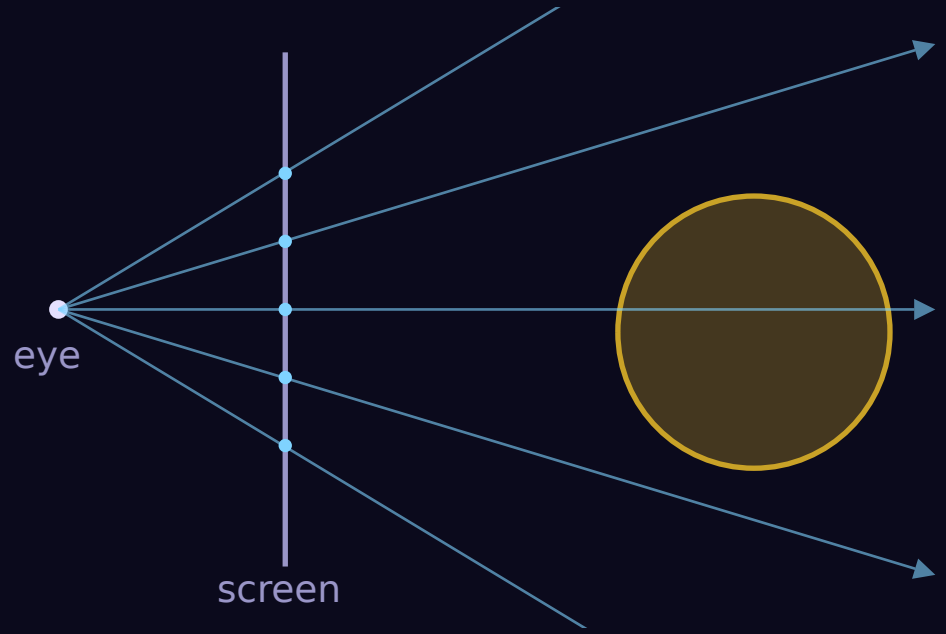
Written in **Futhark**, compiled to **WebGPU**, running in the browser at **1080p 60 fps**.

Free flight, spells, a game controller, hot reload, a WebGL fallback.

Two ways to draw 3D

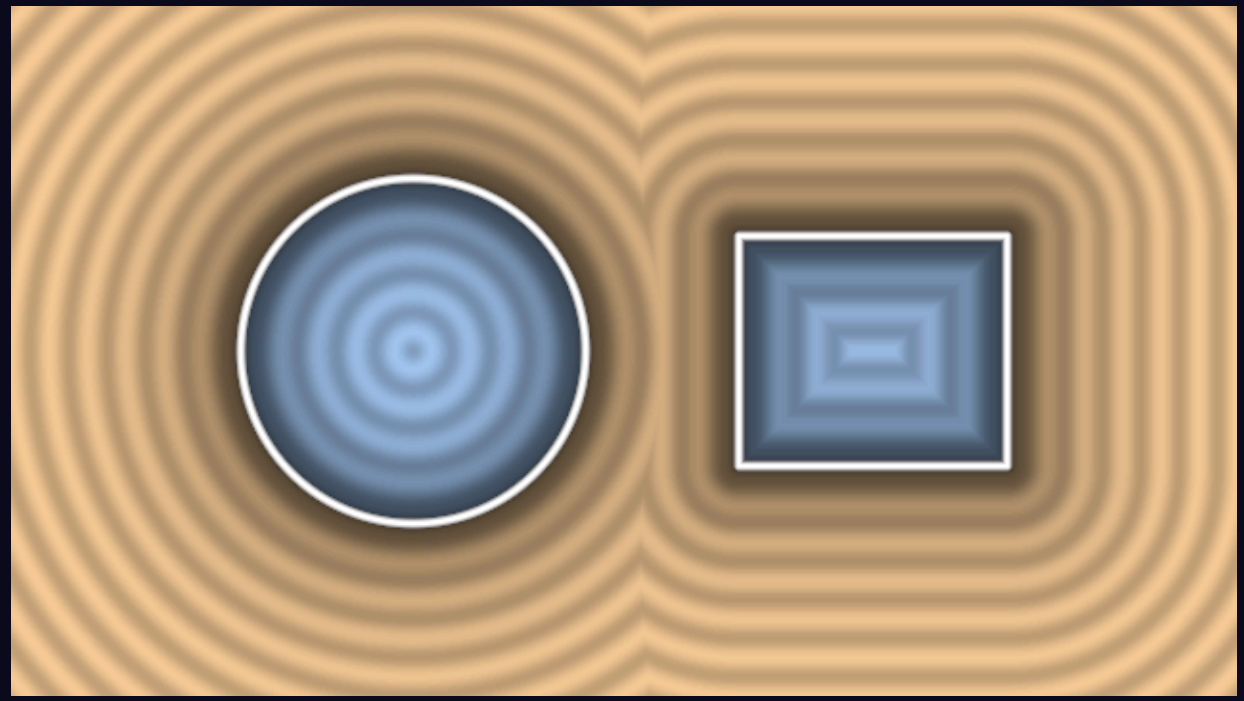


Rasterising: project each triangle's corners onto the screen, fill the pixels between. Shapes must be meshes.



Ray marching: for each pixel, follow a ray from the eye and ask the scene "how far to the nearest surface?" until it is ≈ 0.

Shapes as distance functions



The distance field of a circle and a box: orange outside, blue inside, a band every 0.1, the surface white

A **signed distance function** $d(p)$ gives the distance from p to the nearest surface (negative inside).

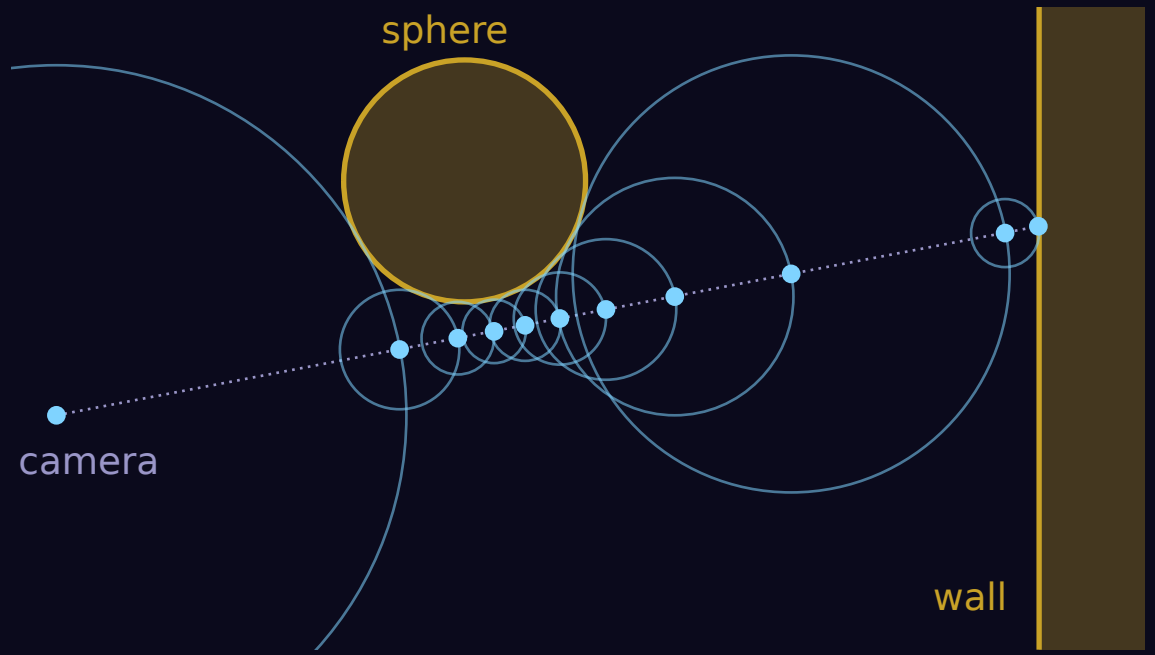
```
def sd_circle p r = length p - r

def sd_box p b =
  let q = abs p - b
  in length (max q 0)
     + min (max q.x q.y) 0

def scene p =
  min (sd_circle (p - c1) 0.45)
      (sd_box (p - c2) (0.35, 0.3))
```

A scene is the **minimum** of its parts.

Sphere tracing

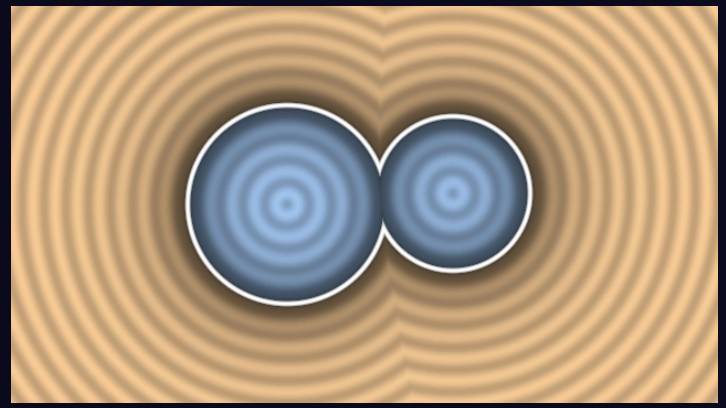


$d(p)$ is a **safe step**: nothing is nearer, so the ray can jump that far without passing through anything.

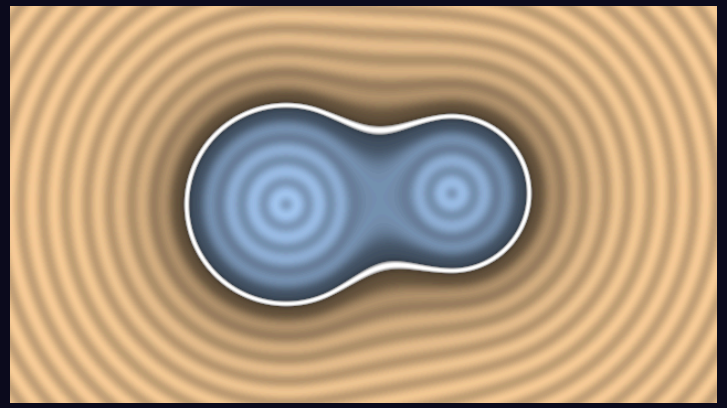
```
loop t = 0 while t < tmax do
  let d = scene (ro + t * rd)
  in if d < eps * t then hit t
     else t + d
```

Big steps in the open; small ones skimming the sphere; then it speeds up again.

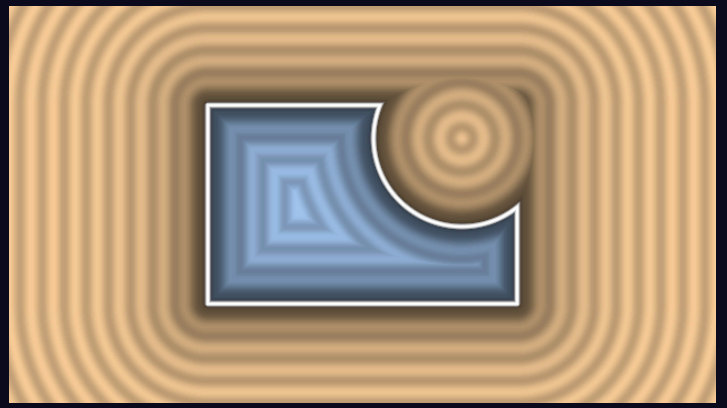
Combining shapes is arithmetic



`min a b`: union



`smin k a b`: smooth union, a fillet of size k



`max a (-b)`: subtract b

```
def smin k a b =
  let h = clamp 0 1 (0.5 + 0.5 * (b - a) / k)
  in mix b a h - k * h * (1 - h)
```

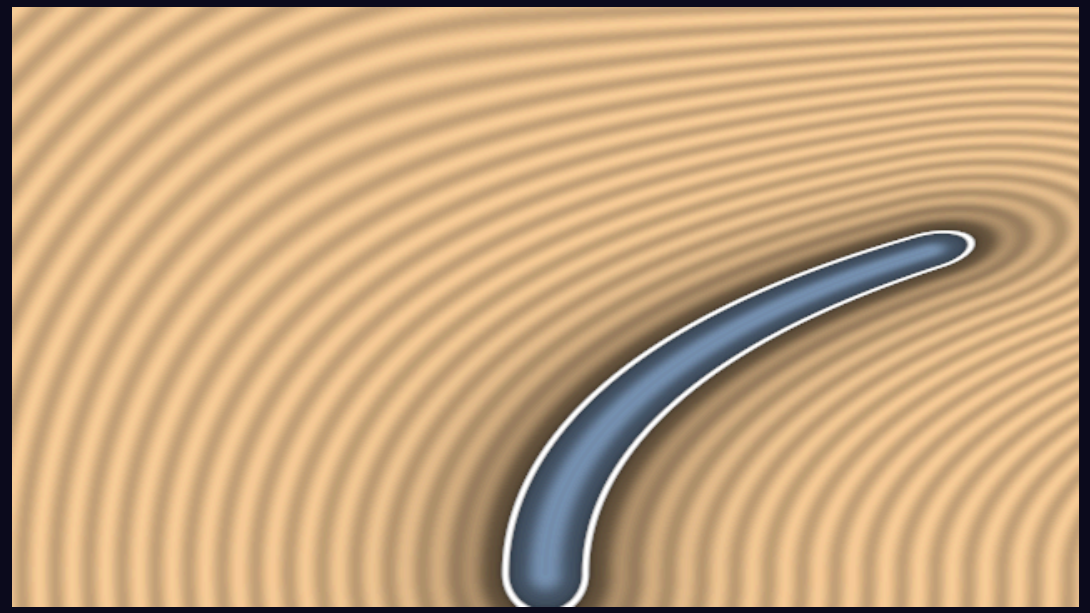
The robe flows into the shoulders, the arms into the robe, the cradle's prongs into its collar: all `smin`.

Transform the point, not the shape



Repetition: fold space into cells (`p - cell * round (p / cell)`), and one circle becomes infinitely many

The rune circle and the orb’s prongs are folded around an axis the same way.



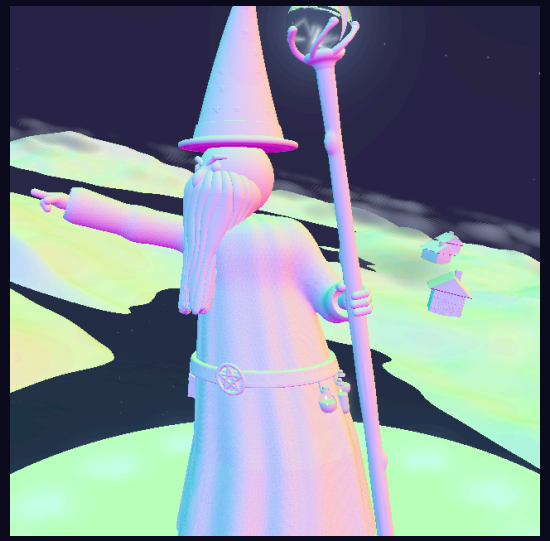
Bending: rotate each point by an angle growing with its height, then ask an ordinary capsule

The hat droops this way. Deforming space stretches distances, so they are scaled down by the worst stretch to keep steps safe.

What each pixel knows



final colour



surface normal (as RGB)



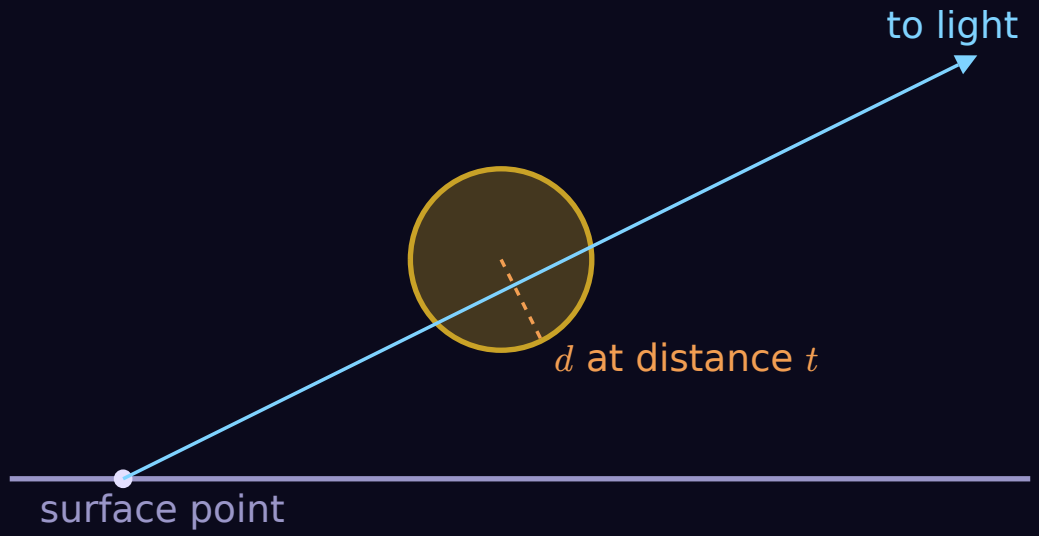
material colour



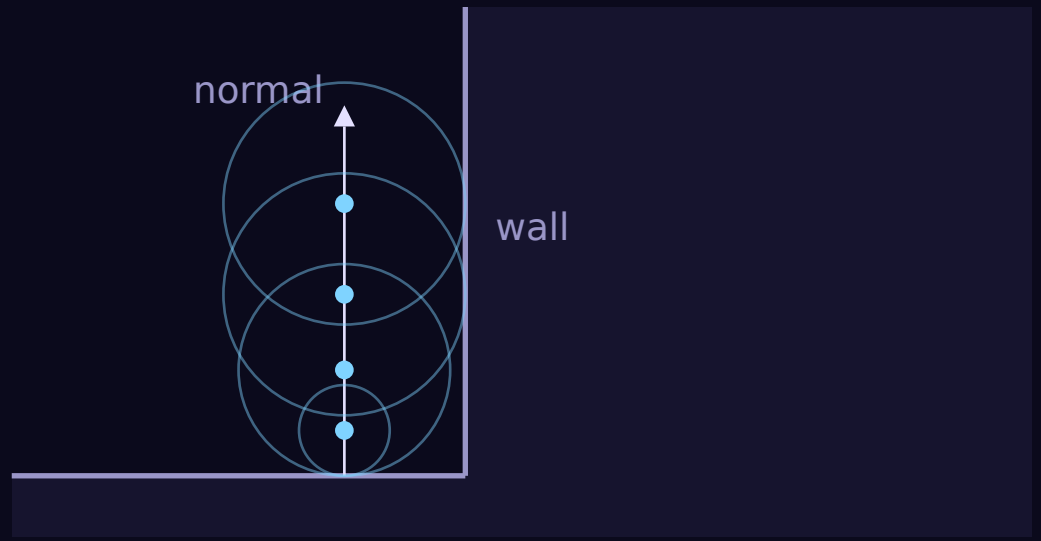
ambient occlusion

The **normal** is the gradient of d: four samples on a tetrahedron around the hit point. **Albedo** comes from the hit part's material and its procedural texture. **AO** asks d how crowded the surroundings are. Lighting combines them per light. These are the drawer's "View" modes, for debugging.

Shadows and occlusion from the same function



Soft shadow: march towards the light. The smallest $k \cdot d/t$ along the way says how close the ray grazed something: 0 is full shadow, near misses give a penumbra.



Ambient occlusion: step out along the normal. Where d at a step is less than the distance stepped (circles cut off by the wall), the corner gets less sky light.

Textures without images



stars, band, brows, eyes and beard locks: hashes, noise and SDFs



bricks: cell coordinates with every other row offset, a hash per brick, the mortar as a **bump**

Bump mapping tilts the normal by a pattern's gradient. **Footprint filtering**: a pixel at distance t is $\approx 0.0008t$ wide (at 1080p); patterns finer than that fade out instead of flickering.

Glass, glow and media



lightning: distance to jagged polylines gives a hot core and a halo



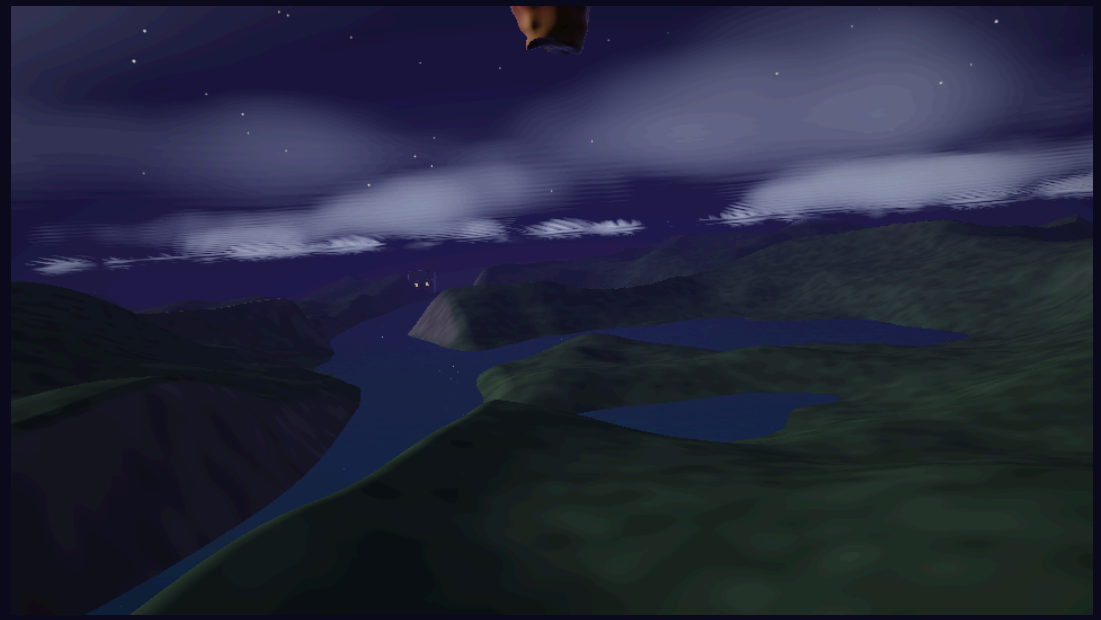
fire: rising noise inside, embers drifting up



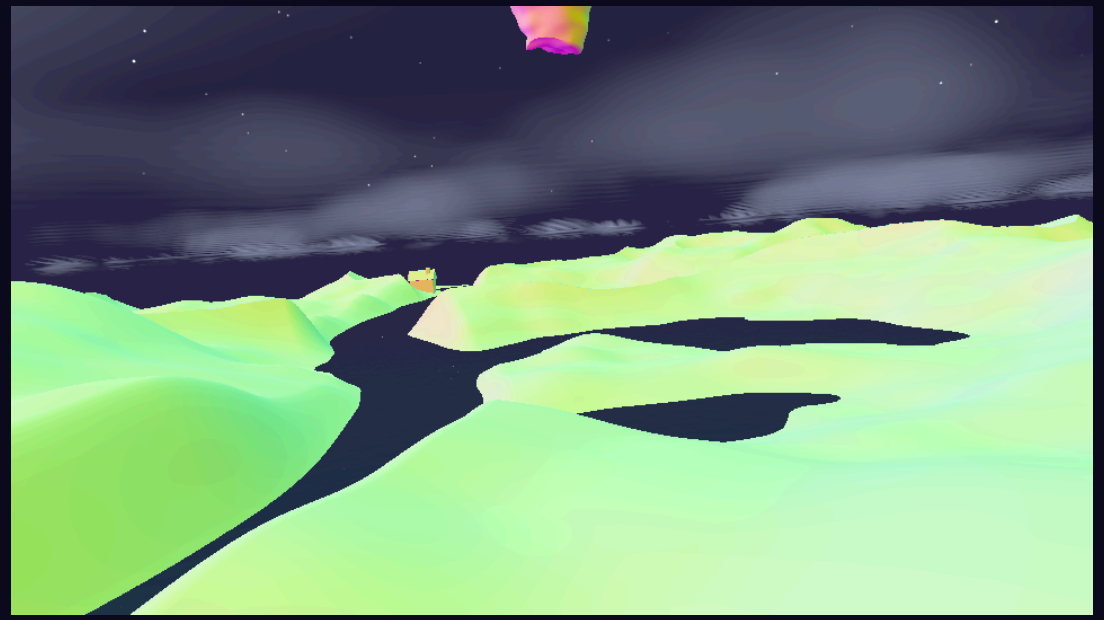
wind: a vortex, and wisps on a tilted ring

The **orb** is solved exactly (ray–sphere) and the ray **refracted** in and out with a **Fresnel** reflection. **Glows** are **media**: each march step adds light $\propto$ a density such as $e^{-d/\,\mathrm{extent}}$; steps are capped near them so none is skipped.

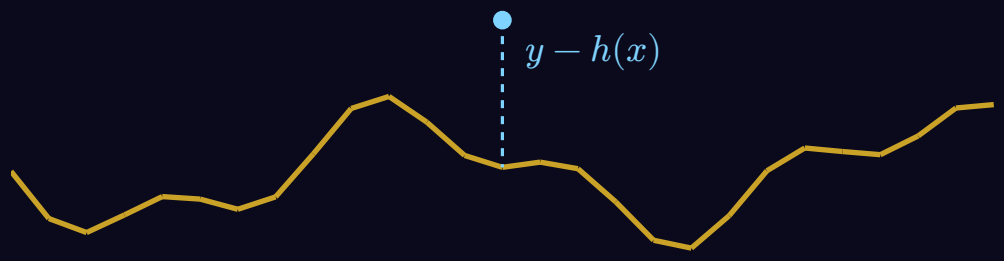
Terrain: a heightfield with care



noise octaves, ridges, carved rivers, a sea plane

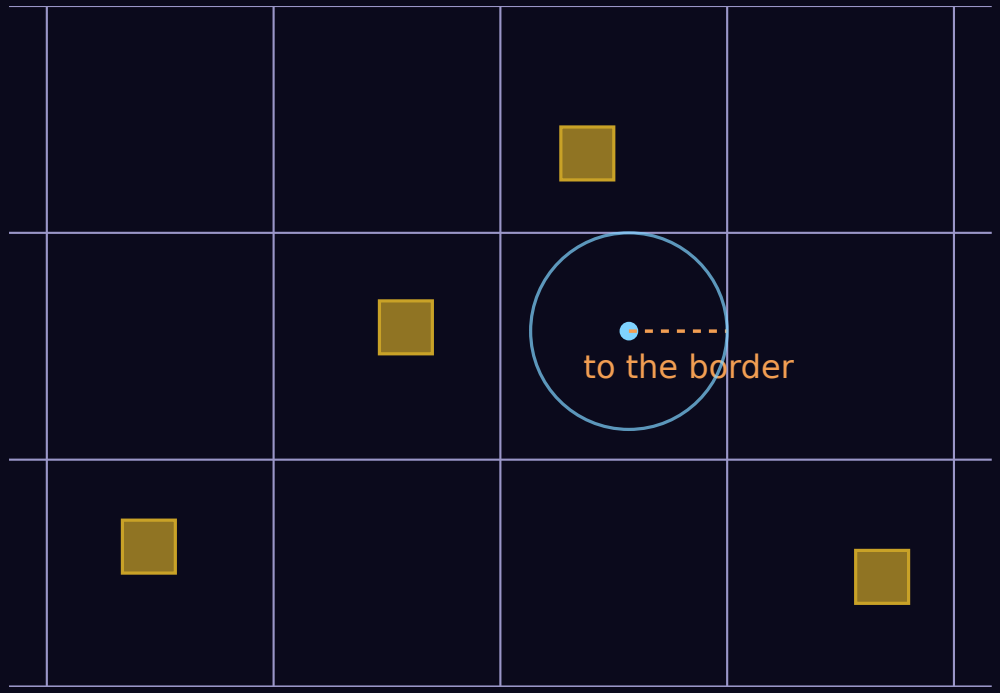


its normals, from the **analytic gradient**: the noise is differentiated alongside its value, one evaluation instead of four



Height above the ground is not a distance on slopes, but the slope is bounded, so $0.6(y - h)$ is a safe step. Far above, a **cheaper bound** (first octaves only) stands in.

Infinite buildings from a hashed grid



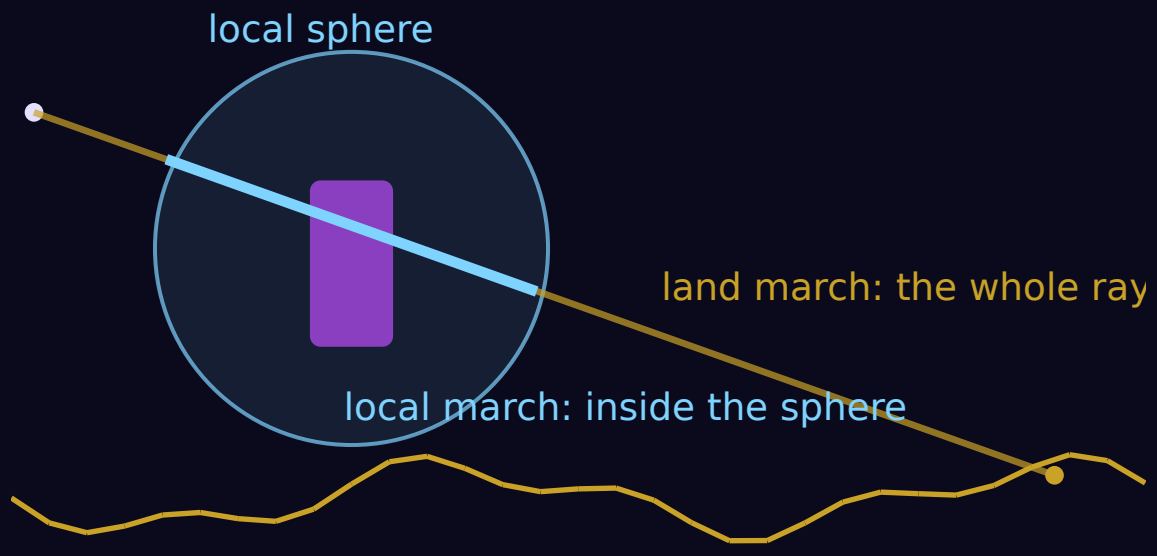
In a 30-unit grid, a hash of the cell decides: a building or not, which, where, how turned.

Marching every cell would be ruinous. Every shortcut returns a **lower bound** on the true distance:

- other cells' buildings stay inside their cells, so they are at least as far as this cell's border (the circle)
- above every roof: the height above the tallest
- the march re-checks buildings only when an earlier bound, less the distance travelled since, could be the nearest

Cottages, towers, pyramids: a few SDF primitives each.

Two marches per ray



Millimetre fingers next to a 150-unit horizon: one march over everything would take tiny steps everywhere.

So the **land** (terrain, water, buildings) and the **local scene** (wizard, rock, glows) are marched separately, each with its own bounds, and the nearer hit wins.

Every part has a **layer** bit, so one scene function serves the camera, shadows (only some parts cast) and debug toggles.

Over-relaxed sphere tracing



Take steps **1.4×** the safe distance. If the next point’s bound shows the step jumped past what the last one allowed (the two spheres don’t overlap), step back and continue normally.

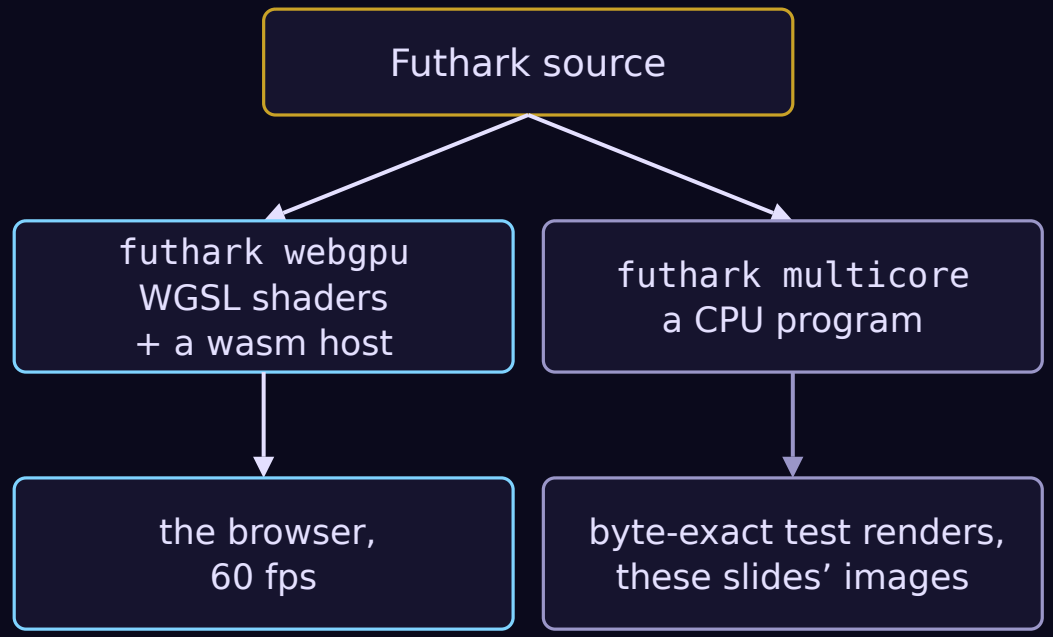
Fewer steps on long, grazing terrain rays. Keinert et al., “Enhanced Sphere Tracing”, 2014. 1.3 and 1.6 measured slower than 1.4 here.

Futhark

A pure, statically typed, array language (think a small ML) that compiles `map`, `reduce`, `scan`, `filter` to GPU kernels.

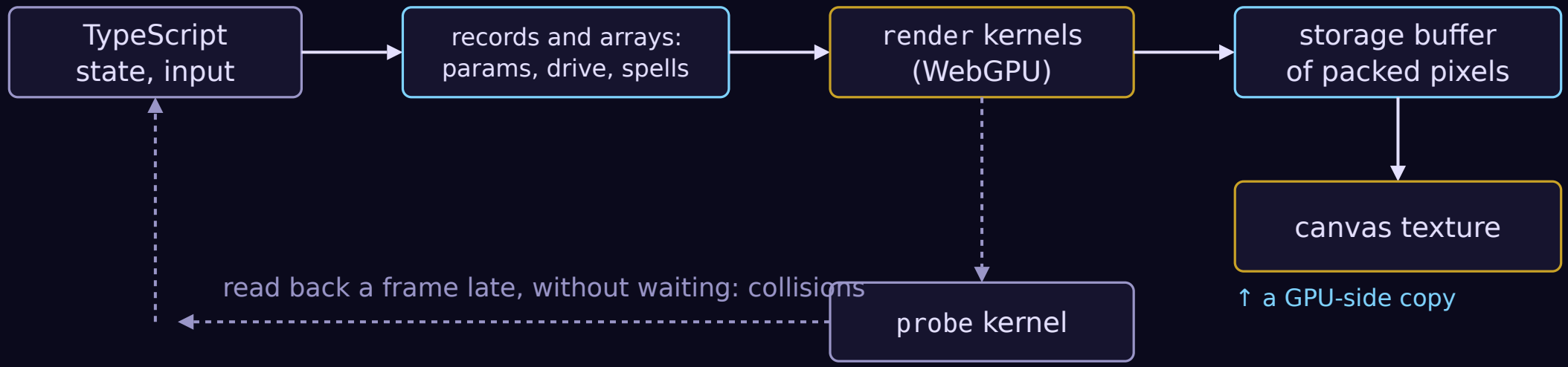
```
def render_frame prm w h ... : []u32 =
  tabulate (w * h) (\i ->
    let (x, y) = (i % w, i / w)
    in pack (pixel prm x y))
```

Records compile to registers: a `vec3` is three floats.



One source, two targets: refactors are checked to change no byte.

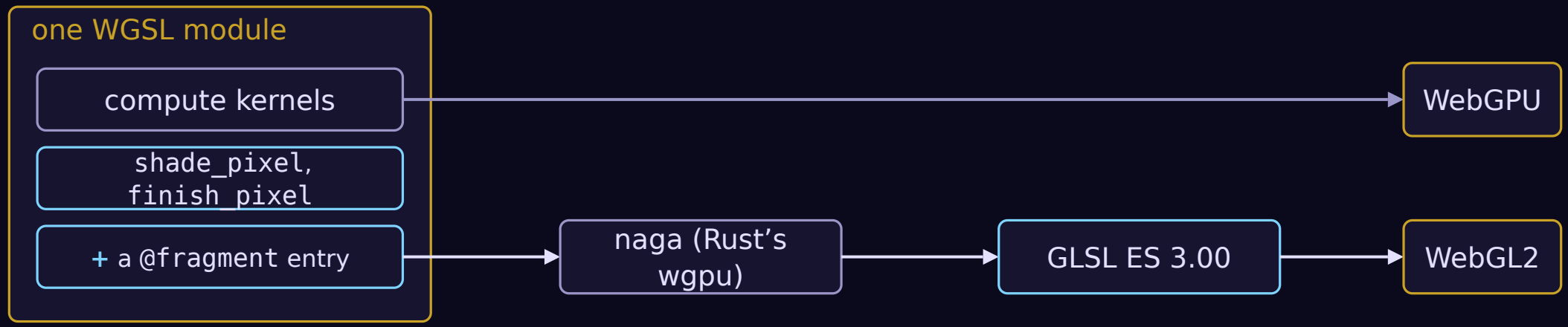
From GPU to screen, with no CPU round trip



Futhark creates its own GPU device; the page captures it (wrapping `requestDevice`) so it can copy Futhark's output buffer straight into the canvas.

All state lives in TypeScript and goes in as data each frame, so the renderer stays a pure function. ≈ 80 drawer parameters, their record type and defaults come from one JSON schema.

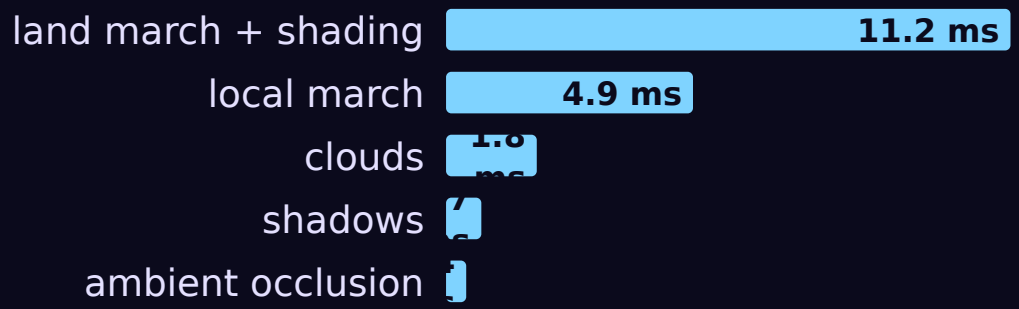
A WebGL fallback from the same program



WebGL has no compute shaders, and Futhark only writes compute kernels. But the per-pixel work sits in two GPU functions taking only scalars, so each build gets a second entry point that calls them from a fragment shader. A compile blocks WebGL; compiling first in a throwaway Worker fills the browser's shader cache, so the real compile then takes milliseconds.

Measuring where time goes

Ablation: build a variant with one feature off, time it.

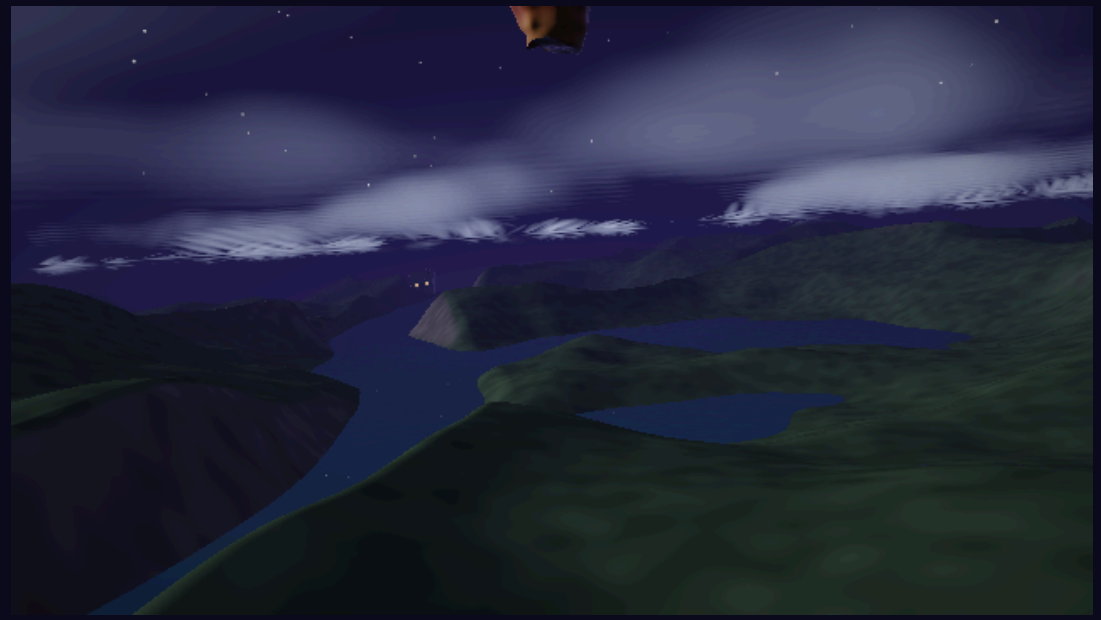


Cost of each part, ms per 1080p frame, out of 18.9 ms (before the last round).

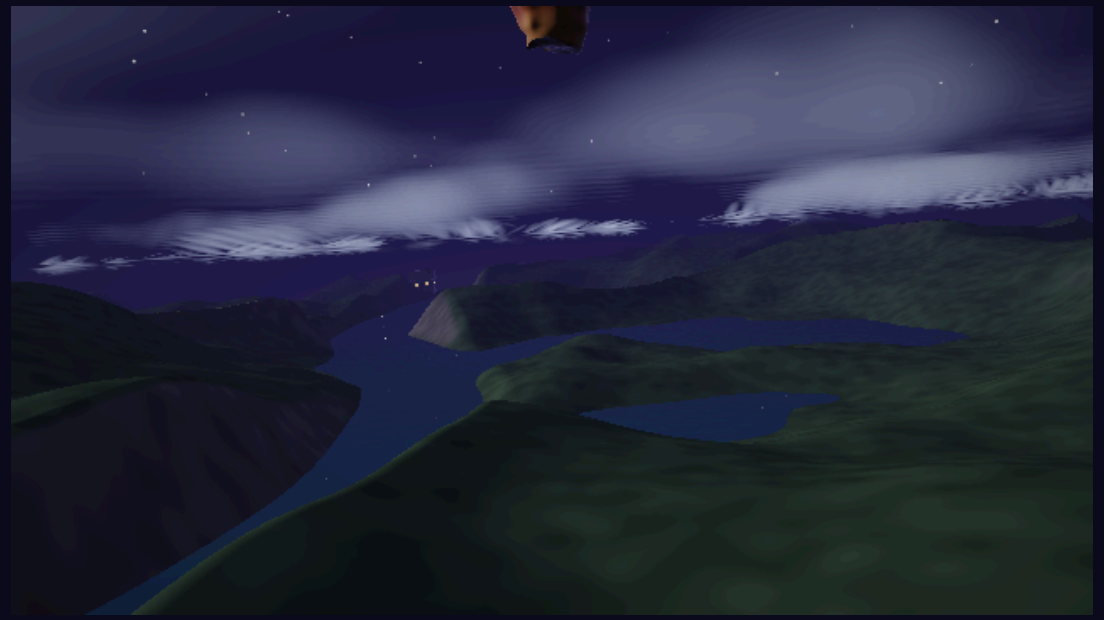
Biggest wins, ms saved:



Trading pixels for milliseconds



terrain hit within $0.0004t$



within $0.0015t$: 1.7 ms faster



A pixel's footprint grows with distance. Finding a surface within ≈ 2 pixels' depth moves no silhouette; it only shifts the shading point slightly.

Shader size is a hidden cost



Chrome compiles each GPU **pipeline** at load, slowly, on one thread. A huge shader meant seconds of black screen.

`#[noinline]` keeps a function as a real GPU function, but it receives every record field as a parameter: passing the whole **pose** means ≈ 200 arguments.

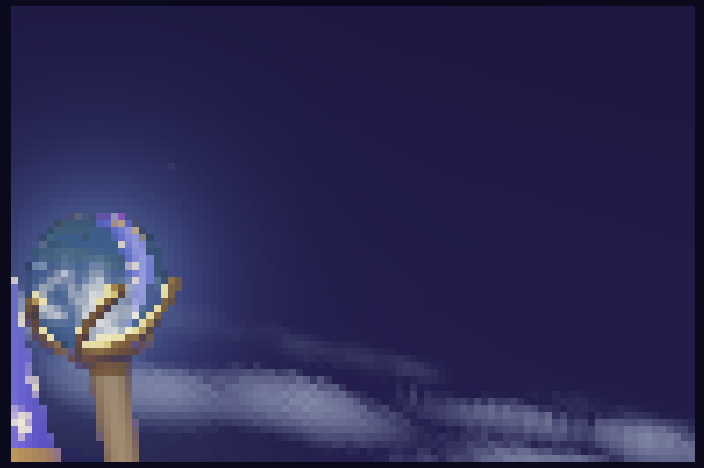
```
-- what the sky needs, out of ~200
type sky_in = { horizon, zenith, moon,
                moon_size, ..., secs }

#[noinline]
def sky_colour (sk: sky_in) rd = ...

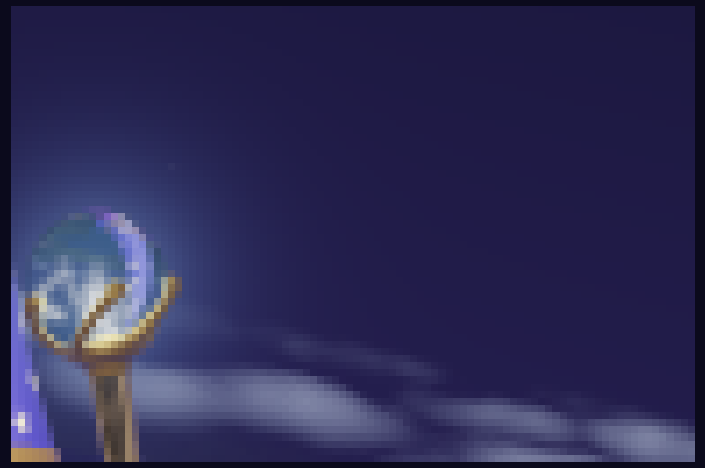
-- callers are unchanged
def sky pose rd =
  sky_colour (sky_in_of pose) rd
```

Plus: fewer kernels holding the shading, and pipelines created asynchronously. Time to first frame roughly halved.

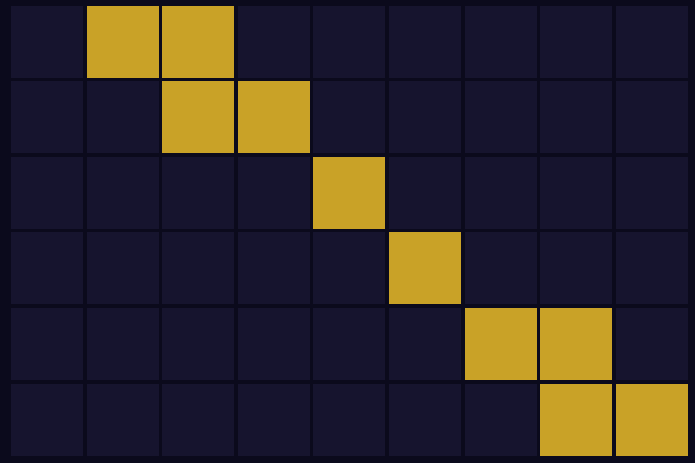
Anti-aliasing as a second pass



one ray per pixel (close up)

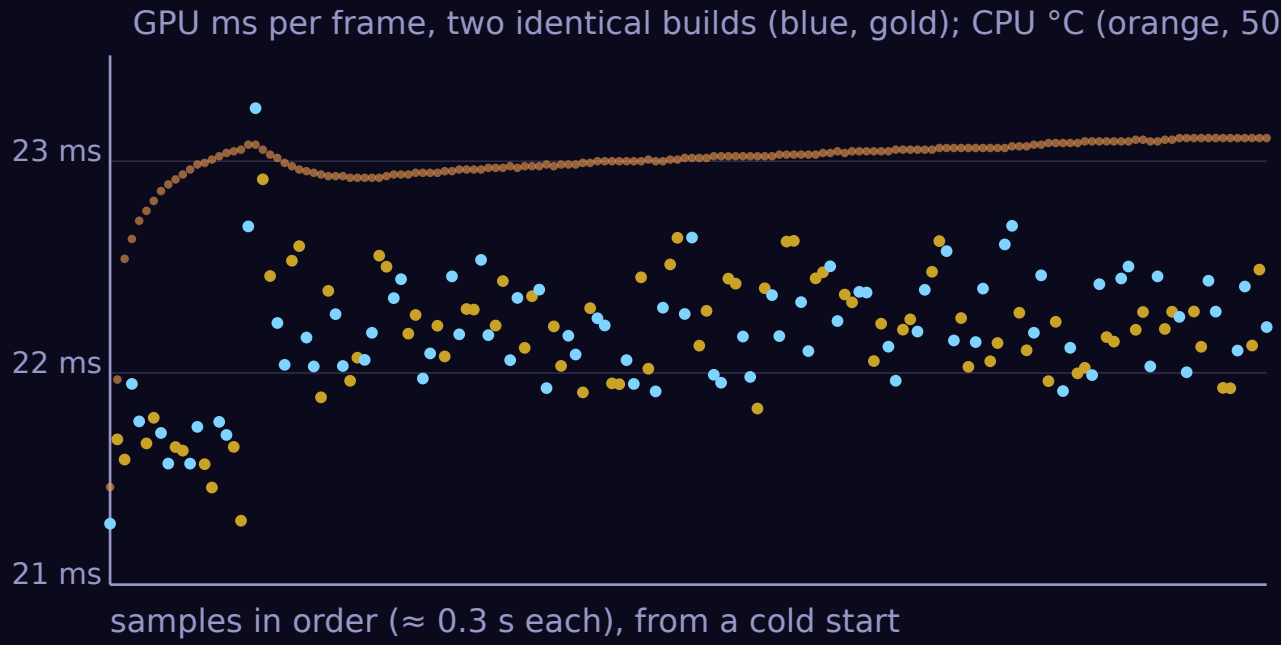


16 rays per pixel, averaged



Edge supersampling: find pixels whose depth jumps or contrast is high, `filter` them into a list, trace 4 more rays for those only (+11 ms). **FXAA** just blurs across detected edges (+0.4 ms).

Benchmarking honestly on a laptop



An **A/A test**: two identical builds. From cold, frames drift ≈ 1 ms slower for a minute as the chip heats, then hold at its power limit. Fans at 100% changed nothing.

So: **warm up**, interleave **A B B A**, report a mean difference with a 95% interval.

to detect	rounds
0.5 ms	≈ 3
0.25 ms	≈ 9
0.1 ms	≈ 55

Bugs found along the way

- **false arriving as true**: the runtime copied a 1-byte bool into a 4-byte slot of uninitialised memory (patched: `malloc` → `calloc`)
- **names containing ‘z’** broke GPU calls (escaped at the call, not the definition; fixed upstream)
- `x ** 2` **for negative** `x` is NaN in WGSL: one explosion ring turned the whole screen white
- **empty arrays** bound as too-small GPU buffers
- **Asyncify’s 4 KB stack** overflowing on a mid-frame read-back
- **Chrome compiling pipelines on its GPU process’s main thread**, freezing other canvases for seconds

Found by comparing CPU and GPU renders, and by bisecting with build variants.

Takeaways

- **One distance function** gives geometry, normals, shadows, occlusion and collisions
- **Correct bounds** are the core of performance: every shortcut must never overestimate the distance
- A **pure renderer** with state passed in is easy to test byte-for-byte
- On the GPU, **code size and kernel count** matter as much as arithmetic
- **Measure**: ablation, timestamps, A/A tests, confidence intervals



github.com/badly-drawn-wizards/wizard-vibes